Lecture 03: Python Basics (Chapter 1)

교재 Ch. 1.4 $\sim$ 1.8

유준수 (Joon Soo Yoo, Ph.D.)

2026년 3월 10일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

1.4 String Basics (문자열 기초)

1.4 String Basics: 문자열이란?

- **String (문자열):** 따옴표로 감싸진 일련의 문자 집합 (Sequence of characters).
- **규칙:** 반드시 짝이 맞는 따옴표(Single ' ' 또는 Double " ")를 사용해야 합니다.

종류	Valid (올바름)	Invalid (틀림)
일반 문자열	`"Hello!"`	`Hello!` (변수로 인식됨)
숫자 문자열	`"17"`	`17` (정수로 인식됨)
따옴표 혼용	`"Where?"`	`"Where?'` (짝이 안 맞음)

"따옴표 안에 들어가면 더 이상 숫자가 아닙니다!"

파이썬에게 `123`은 계산기 속 숫자지만, `"123"`은 그저 세 글자의 텍스트일 뿐입니다.

따옴표 안의 따옴표 (Quote Nesting)

문자열 안에 따옴표를 포함시키고 싶을 때는 서로 다른 종류의 따옴표를 중첩해서 사용합니다.

문법 가이드

- **작은따옴표 포함:** 전체를 큰따옴표로 감쌈 → `"Won't this work?"`
- **큰따옴표 포함:** 전체를 작은따옴표로 감쌈 → `'He said "Hi"'`

Bad Case: 같은 따옴표 중첩

`"He said "Hi""` → 파이썬은 중간의 `"Hi"`를 문자열의 끝으로 착각하여 에러를 냅니다.

len() 함수: 문자열의 길이 측정

`len()` 함수는 문자열에 포함된 **문자의 개수**를 반환합니다.

- **공백 포함:** 띄어쓰기 한 칸도 엄연히 한 글자로 카운트됩니다.
- **Empty String:** `""` (아무것도 없는 문자열)의 길이는 `0`입니다.

Code Example

```
name = "Hi Ali"
print(len(name))        # 출력: 6 (공백 포함)

number = "12"
print(len(number))      # 출력: 2 (숫자가 아니라 글자로 인식)
```

문자열 결합 (Concatenation)

+ 연산자를 사용하여 여러 문자열을 하나로 길게 연결할 수 있습니다.

- **주의:** 숫자와 문자열의 연산은 다릅니다.
  - `1 + 0` → `1` (더하기)
  - `"1" + "0"` → `"10"` (이어붙이기)

실전 결합 코드

```
first = "Alan"
last = "Turing"
full_name = first + " " + last
print("Welcome, " + full_name + "!") # Welcome, Alan Turing!
```

1.5 Number Basics (숫자 기초)

1.5 Number Basics: 숫자 자료형 (Data Types)

파이썬은 숫자를 크게 두 가지 형식으로 구분하여 저장합니다.

- **Integer (int, 정수):** 소수점이 없는 정수 (예: `1`, `-5`, `100`)
- **Floating-point (float, 부동 소수점):** 소수점이 포함된 실수 (예: `2.0`, `3.14`, `-0.5`)

type() 함수로 확인하기

`x = 1` → `type(x)`는 `<class 'int'>`

`y = 2.0` → `type(y)`는 `<class 'float'>`

`s = "32"` → `type(s)`는 `<class 'string'>` (주의!)

"따옴표 안에 있는 "12.0"은 숫자가 아니라 문자열입니다."

기본 산술 연산자 (Arithmetic Operators)

파이썬은 사칙연산을 위해 다음과 같은 기호를 사용합니다.

연산자	설명	예시	결과
+	더하기 (Addition)	`3 + 2`	5
-	빼기 (Subtraction)	`7 - 2`	5
*	곱하기 (Multiplication)	`3 * 4`	12
/	**나눗셈 (Division)**	`10 / 2`	`5.0`

나눗셈의 결과

파이썬에서 / 연산자를 사용하면 결과가 정수로 딱 떨어지더라도 항상 **float(실수)** 형식을 반환합니다. (예: `10 / 2 = 5.0`)

연산 우선순위 (Operator Precedence)

여러 연산자가 섞여 있을 때, 파이썬은 정해진 우선순위에 따라 계산합니다.

1. **괄호** `()`**:** 가장 높은 우선순위
2. **지수 연산** `**`**:** 거듭제곱 (예: `2 ** 3 = 8`)
3. **단항 연산** `+`, `-`**:** 양수/음수 표시 (예: `-4`)
4. **곱셈/나눗셈** `*`, `/`
5. **덧셈/뺄셈** `+`, `-`

연습 문제

$4 * 3 ** 2 + 1 = 4 * 9 + 1 = 36 + 1 = 37$

우선순위의 함정 (Precedence Pitfalls)

- **음수의 제곱:** `-4 ** 2`의 결과는 무엇일까요?
  - 파이썬은 지수(`**`)를 음수 기호(`-`)보다 먼저 처리합니다.
  - 즉, `-(4 ** 2)`가 되어 결과는 `-16`이 됩니다.
  - 16을 얻고 싶다면 `(-4) ** 2`와 같이 괄호를 써야 합니다.
- **정수와 실수의 혼합 연산:**
  - 정수와 실수가 연산하면 결과는 항상 **실수**가 됩니다.
  - `2 * 1.5 = 3.0`

1.6 Error Messages (에러와 친해지기)

1.6 Error Messages: 파이썬의 대화 시도

에러는 실패가 아니라, 컴퓨터가 당신의 명령을 이해하지 못해 보내는 **'도움 요청'** 입니다. 전문적인 개발자는 에러 메시지에서 다음 세 가지를 먼저 확인합니다.

- **Line Number:** 에러가 발생한 위치 (몇 번째 줄인가?)
- **Error Type:** 어떤 종류의 문제인가? (Ex: `SyntaxError`)
- **Caret (^):** 파이썬이 "여기쯤에서 이상함을 느꼈어"라고 가리키는 지점

Traceback 예시

```
File "example.py", line 2
print "Hello"
^
SyntaxError: Missing parentheses...
```

주요 에러 삼총사 (Common Error Types)

에러 타입	원인	해결책
`SyntaxError`	문법 규칙 위반 (따옴표, 괄호 누락)	오타 확인 및 짝 맞추기
`NameError`	정의되지 않은 변수/함수 이름 사용	이름 스펠링 확인 (`print` vs `prints`)
`IndentationError`	들여쓰기(Space/Tab) 실수	줄 시작 부분의 공백 제거

팁

실제로 로직을 짤 때 발생하는 복잡한 버그도 결국 이 기초적인 메시지를 꼼꼼히 읽는 것에서 해결이 시작됩니다.

1.7 Comments (주석과 코드 품질)

1.7 Comments

주석(#)은 프로그램 실행에는 영향을 주지 않지만, 코드를 읽는 사람(특히 3개월 뒤의 나)을 위해 남기는 설명입니다.

- **Formatting:** # 뒤에 한 칸의 공백을 두는 것이 권장됩니다. (예: `# Good`)
- **Good Practice:** "무엇을(What)" 하는지보다 **"왜(Why)"** 하는지를 설명하세요.

주석 활용법

```
# Get user's temperature in Celsius

temp = float(input())   # Unit conversion factor
```

코드 품질: 가독성 높이기

좋은 프로그래머는 코드를 쓰는 시간보다 **읽는 시간**이 더 많다는 것을 압니다.

- **Blank Lines:** 서로 다른 역할을 하는 코드 블록 사이에는 빈 줄을 넣어주세요.
- **Docstring (`"""...."""`):** 프로그램 최상단에 작성하며, 코드의 목적과 제작자 정보를 담습니다.

전문적인 코드 구조 예시

```
"""
Author: Joon Soo Yoo
Description: This program calculates gravity effects.
"""
# Input section
mass = float(input("Mass: "))

# Calculation section
force = mass * 9.8
```

Any Questions?